Public API

Status	Done
Assign	🌌 Pasit Promchawee
Classes	🧑‍💼 Live 15 - Essential Python for Data Analyst

Let's use API in Python.

I will use Star Wars API for this one because I'm so lazy to find free API.

https://swapi.info/

And here's all my exercises that I will do for this homework!

Libraries

```
import requests
import time
import json
import random
import pandas as pd
import numpy as np
```

Exercise #1: Get Character Data

```
characters = []

for i in range(7):
  url = f'https://swapi.info/api/people/{i+1}'
```

```
  response = requests.get(url)
  response_json = response.json()
  data = [response_json['name'], response_json['height'], res
ponse_json['mass']]
  characters.append(data)
  print(f'Success: {i+1}')
  time.sleep(1)

print(characters)
```

```
characters = []

for i in range(7):
  url = f'https://swapi.info/api/people/{i+1}'
  response = requests.get(url)
  response_json = response.json()
  data = [response_json['name'], response_json['height'], response_json['mass']]
  characters.append(data)
  print(f'Success: {i+1}')
  time.sleep(1)

print(characters)
```

```
Success: 1
Success: 2
Success: 3
Success: 4
Success: 5
Success: 6
Success: 7
[['Luke Skywalker', '172', '77'], ['C-3PO', '167', '75'], ['R2-D2', '96', '32'], ['Darth Vader', '202', '136'], ['Leia Organa',
```

This code only focus on getting all characters data with `name`, `height` and `mass`.

Exercise #2: Random Character Data

```
random_index = random.randint(1, 83)

url = f'https://swapi.info/api/people/{random_index}'
response = requests.get(url)
response_json = response.json()
data = [response_json['name'], response_json['height'], respo
nse_json['mass']]
```

```
print(f'Character: {random_index}')
print(data)
```

```
random_index = random.randint(1, 83)

url = f'https://swapi.info/api/people/{random_index}'
response = requests.get(url)
response_json = response.json()
data = [response_json['name'], response_json['height'], response_json['mass']]

print(f'Character: {random_index}')
print(data)
```

```
Character: 2
['C-3PO', '167', '75']
```

This code is similar to the first one.

Just it only returns 1 character and it's random.

It's random between 1 to 83. This one is hard coded. Probably, I'll make it more dynamic later.

Exercise #3: Tabular Data of Planets

```
url = "https://swapi.info/api/planets"
response = requests.get(url)
response_json = response.json()

planets_df = pd.DataFrame(response_json)

print(f"Total planets retrieved: {len(planets_df)}")
display(planets_df.head())
```

```python
url = "https://swapi.info/api/planets"
response = requests.get(url)
response_json = response.json()

planets_df = pd.DataFrame(response_json)

print(f"Total planets retrieved: {len(planets_df)}")
display(planets_df.head())
```

```
Total planets retrieved: 60
```

	name	rotation_period	orbital_period	diameter	climate	gravity	terrain	surface_water	population	
0	Tatooine	23	304	10465	arid	1 standard	desert	1	200000	[https://swapi.info
1	Alderaan	24	364	12500	temperate	1 standard	grasslands, mountains	40	2000000000	[https://swapi.info
2	Yavin IV	24	4818	10200	temperate, tropical	1 standard	jungle, rainforests	8	1000	
3	Hoth	23	549	7200	frozen	1.1 standard	tundra, ice caves, mountain ranges	100	unknown	
4	Dagobah	23	341	8900	murky	N/A	swamp, jungles	8	unknown	

This one I'm getting all data of planets.

And turn to data frame with `pandas` library.

Exercise #4: Planet Aggregation

```python
planets_df.groupby('climate')['climate'].count().sort_values(ascending=False).head(10)
```

```
planets_df.groupby('climate')['climate'].count().sort_values(ascending=False).head(10)
```

climate	climate
temperate	23
unknown	12
hot	3
arid	3
arid, temperate, tropical	2
frigid	2
arid, rocky, windy	1
artificial temperate	1
hot, humid	1
murky	1

dtype: int64

This one I group by `climate` and count how many `climate` in this dataset have?

```
planets_df_clean = planets_df.query("population != 'unknow
n'").copy()
planets_df_clean['population'] = pd.to_numeric(planets_df_cle
an['population'])

planets_df_clean.groupby('name')['population'].sum().sort_val
ues(ascending=False).head(10)
```

```python
planets_df_clean = planets_df.query("population != 'unknown'").copy()
planets_df_clean['population'] = pd.to_numeric(planets_df_clean['population'])

planets_df_clean.groupby('name')['population'].sum().sort_values(ascending=False).head(10)
```

name	population
Coruscant	1000000000000
Skako	500000000000
Geonosis	100000000000
Mon Cala	27000000000
Eriadu	22000000000
Sullust	18500000000
Nal Hutta	7000000000
Muunilinst	5000000000
Naboo	4500000000
Ord Mantell	4000000000

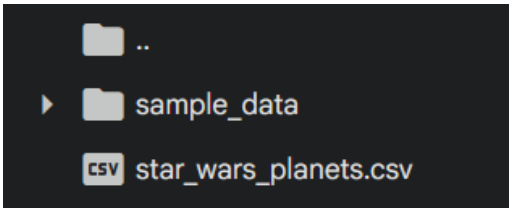
dtype: int64

I try to aggregate `population` field.

But it needs to be cleaned because the empty value is `"unknown"` and the field data type is `string`.

I use `filter` to clear `"unknown"` text (I suppose to replace more than filter) and then convert to `numeric`.

Then I group by `name` and sum the `population`.

Exercise #5: Export Data to CSV

```python
planets_df.to_csv('star_wars_planets.csv')
```